



Familiarização com CORE Network Emulator e Wireshark

(Duração: 2 horas)

O objetivo deste trabalho é a familiarização com as ferramentas **CORE Network Emulator** e **Wireshark**¹, que serão utilizadas ao longo dos trabalhos práticos da unidade curricular. Siga os guiões da Secção 1 e da Secção 2, a seguir responda as questões da Secção 3.

1. Emulador CORE

O CORE (*Common Open Research Emulator*) é uma ferramenta para emular redes de computadores. É constituído por uma interface gráfica, que pode ser usada para estabelecer e configurar diferentes topologias de rede, e por um conjunto de módulos escritos em Python, responsáveis pela emulação de várias tecnologias e protocolos de comunicações. Tem como principais vantagens permitir a execução de “código real” nos equipamentos emulados, interligar as topologias de rede emuladas com redes reais, ser eficiente, escalável e simples de usar. Para mais informações sobre o CORE verifique a página de documentação, disponível em: <https://coreemu.github.io/core/index.html>.

Esta ferramenta funciona apenas em FreeBSD e Linux. Uma alternativa à instalação nativa de um sistema consiste em usar um ambiente virtualizado, através de uma máquina virtual ou um contentor. A equipa docente disponibilizou uma versão contentorizada do CORE v9.0.3 ² para correr em ambiente Docker. Pode-se correr o contentor a partir do script e informações do README disponíveis em <https://github.com/vinicf/IRC/> diretamente no Linux, macOS, ou no ambiente WSL do Windows. Os pré-requisitos para a utilização são:

- Em Linux é necessário instalar a Docker Engine.
 - <https://docs.docker.com/engine/install/>

¹ <https://www.nrl.navy.mil/Our-Work/Areas-of-Research/Information-Technology/NCS/CORE/>

² <https://github.com/coreemu/core/tree/release-9.0.3>



- Em Windows deve-se ter o WSL v2 (distribuição Ubuntu) e o Docker Desktop instalados.
 - <https://learn.microsoft.com/pt-br/windows/wsl/install>
 - <https://docs.docker.com/desktop/features/wsl/>
- Em macOS deve-se instalar o Docker Desktop e o Xquartz.
 - <https://docs.docker.com/desktop/setup/install/mac-install/>
 - <https://www.xquartz.org/>
 - Após a instalação do Xquartz, vá a X11 Preferences > Security e habilite a opção "Allow connections from network clients".

1.1. Familiarização com o CORE

O CORE permite criar cenários de redes virtuais recorrendo a *Linux Network Namespaces*³, possibilitando a emulação de topologias e protocolos de rede.

Na Figura 1 é possível observar a interface gráfica do CORE e a localização das ferramentas de criação de nós e ligações.

1.1.1. Tipos de container nodes

Quantos tipos de *Container Nodes* existem no CORE? - Qual o propósito de cada tipo de nó?

1.1.2. Criação de uma topologia simples

Crie uma topologia com **dois nós PC**, ligados entre si através da ferramenta **Link**, semelhante à Figura 2.

Na topologia criada é possível verificar que ambos possuem um endereço **IPv4** e um endereço **IPv6**.

Quais os endereços IPv4 e IPv6 atribuídos a cada nó?

³ https://man7.org/linux/man-pages/man7/network_namespaces.7.html

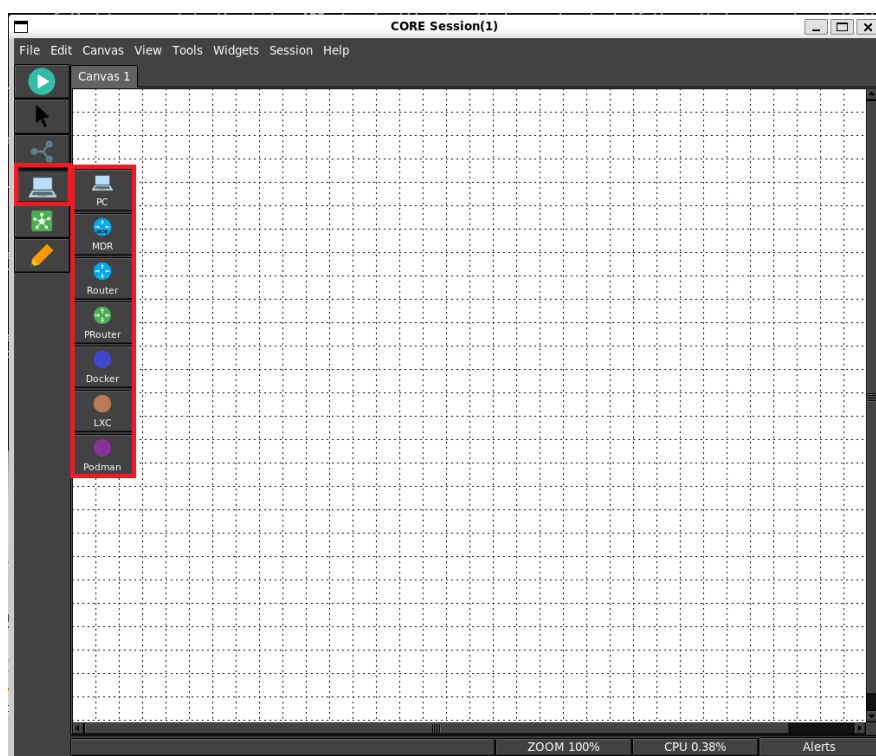


Figura 1 - Interface gráfica do CORE.

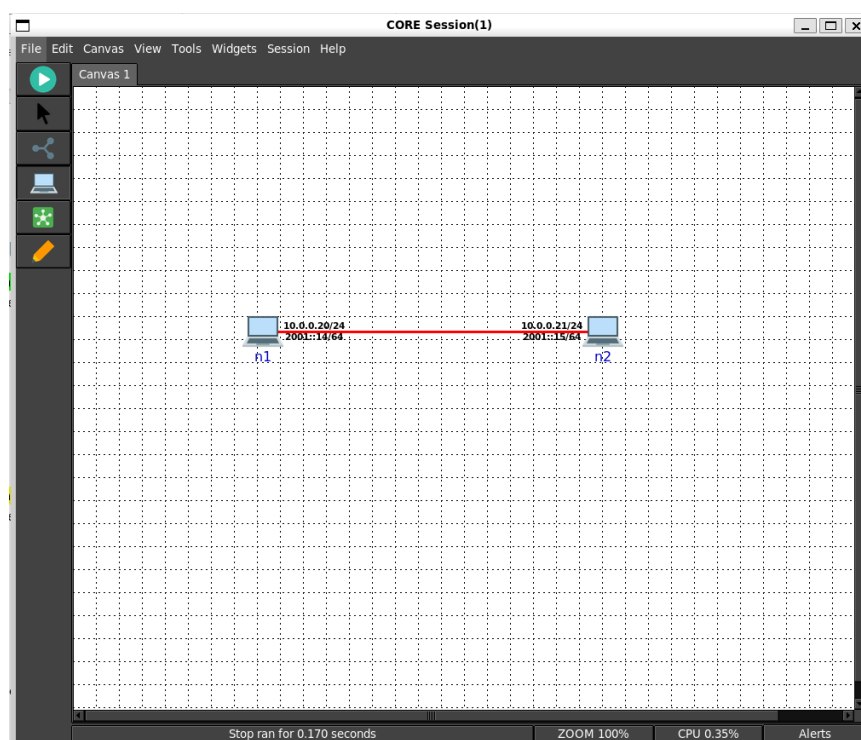


Figura 2 - Topologia exemplo.



1.1.3. Configuração das ligações e dos nós

Um aspeto importante da emulação é a configuração das ligações entre os nós.

Clique com o botão direito sobre a ligação entre os nós e selecione a opção **Configure**.

Será apresentada uma janela, como na Figura 3, com várias opções de configuração.

Quais são as opções de configuração disponíveis para a ligação e qual o propósito de cada uma?

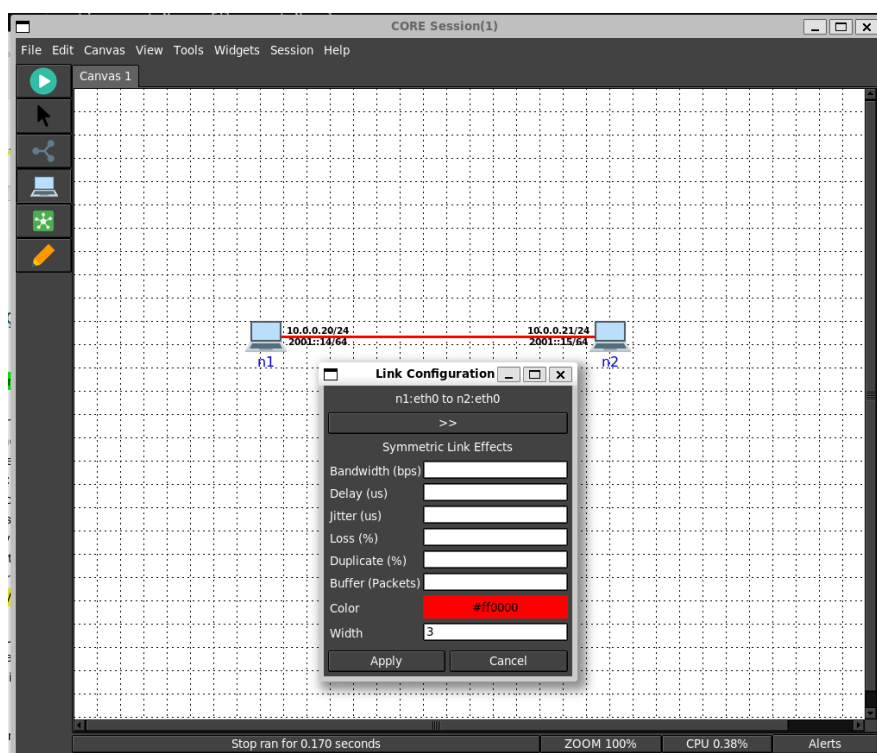


Figura 3 - Configuração das ligações.

O mesmo tipo de configuração pode ser aplicado diretamente aos nós. Abra a janela de configuração do nó e verifique o que pode ser alterado. Altere o endereço IP do nó n1 para 10.0.0.13/24.

1.1.4. Modo de execução

Até aqui estávamos no modo de edição, onde é possível acrescentar nós, ligações e configurá-los. Agora vamos iniciar a emulação da rede em si, no modo de execução. Carregue no ícone **Start Session**. A partir desse momento, já não é mais possível adicionar/remover nós e ligações, ou fazer novas configurações.



No modo de execução, os nós do tipo **Container Nodes** comportam-se de forma semelhante a um contentor. É possível correr comandos a partir deles ao se abrir um terminal (duplo clique sobre o nó) e cada nó possui uma diretoria home própria. Para verificar a diretoria atual, corra o comando *pwd* (print working directory).

Abra o terminal do nó n1 e corra o comando *ip a*. O que faz esse comando? O resultado do comando corresponde à informação apresentada na interface gráfica do CORE?

Agora corra o comando *ping <IP_n2>*. Substitua *<IP_n2>* pelo endereço IPv4 do nó n2. Qual o output apresentado pelo comando *ping* e o seu significado?

O comando **iperf3** é utilizado para medir o débito de uma ligação de rede. No terminal do nó n2, execute *iperf3 -s*. Este nó ficará à escuta de conexões na porta 5201. Agora, no terminal do nó n1, execute o comando *iperf3 -c <IP_n2>*.

Qual o output do comando *iperf3* e o significado dos valores apresentados?

Agora encerre a sessão e volte ao modo de edição. Salvar a topologia (nós e ligações configurados) para execução futura em Files > Save As..., dê o nome do ficheiro de Exemplo-Ficha1 e salve com o formato xml. Verifique e aponte a diretoria em que está a salvar o ficheiro.

1.1.5. Acesso ao contentor e ficheiros dos Container Nodes

Agora vamos a partir da máquina host (Linux/Windows/macOS) aceder ao contentor e verificar como é possível armazenarmos os outputs dos comandos e acedermos a esses outputs a partir do host.

Um contentor é um ambiente virtualizado que se abstrai dos recursos do sistema, possui uma visão própria da árvore de processos e do sistema de ficheiros. Portanto, devemos fazer um acesso remoto ao contentor para aceder o seu ambiente interno. É possível aceder a partir do comando Docker:

```
docker exec -it core /bin/bash
```



Este permite executar processos dentro de um contentor, sendo particularmente útil no nosso caso, quando é necessário interagir diretamente com o ambiente interno para além da interface gráfica do CORE.

O comando estabelece uma sessão interactiva (-it) no container com o nome **core**, iniciando um interpretador de comandos Linux (terminal) *shell bash*. Assim, a partir deste ponto, tem-se acesso completo ao sistema de ficheiros e aos processos em execução dentro do contentor, permitindo realizar configurações ou executar comandos.

O comando `pwd` pode ser utilizado para verificar a diretoria atual, o `ls` (list directory contents) para listar o que há nessa diretoria e o comando `cd` (change directory) para navegar pela árvore do sistema de ficheiros.

Vá até a diretoria raiz `/` e verifique o que há nela. Utilize a opção `-l` no comando `ls` para ver no modo lista longa, assim é possível ver mais detalhes do conteúdo da diretoria. Veja se encontra uma diretoria chamada `shared`.

Na script de execução do contentor **core** disponibilizada, cria-se um volume partilhado entre Docker e host. A diretoria `/shared/` do contentor `core` é sincronizada com a diretoria `/home/<username>/IRC/shared/` do host. Ou seja, podemos nos utilizar dessa diretoria para partilhar os ficheiros entre contentor e host. Mova o ficheiro da topologia `Exemplo-Ficha1.xml` para a diretoria `shared` com o comando `mv <dir_origem>/<filename> <dir_destino>` e verifique que este ficheiro é visível a partir do host na diretoria `/home/<username>/IRC/shared`.

Uma vantagem de utilizar o volume é que o seu conteúdo não é efémero, isto é, se fecharmos a aplicação CORE, o contentor será terminado, mas o conteúdo da diretoria `/shared/` permanece disponível no host em `/home/<username>/IRC/shared/`. Feche a interface gráfica do core e verifique que o ficheiro `Exemplo-Ficha1.xml` permanece disponível.



Agora, ao se correr a script `run-docker.sh` novamente, o novo contentor **core** terá o conteúdo de `/shared/` sincronizado, portanto a topologia salva estará disponível no contentor. Vá a `File > Open...` e abra a topologia `Exemplo-Ficha1.xml`.

Apesar do nome Container Nodes, os nós CORE não são exatamente contentores, eles utilizam-se da ferramenta Linux namespaces para abstrair alguns recursos do sistema e isolar seus processos, porém o sistema de ficheiros, por exemplo, ainda é partilhado entre todos os nós e o sistema que corre o CORE. Ou seja, a diretoria home do nó n1, (E.x.: `/tmp/pycore.1/n1.conf/`) é visível a partir do nó n2 e todos os ficheiros são montadas na diretoria `/tmp` do contentor **core**.

Podemos nos aproveitar desse facto, que o sistema de ficheiros dos Container Nodes é o mesmo do sistema que a aplicação CORE corre (em nosso caso, o contentor **core**), para salvarmos os logs e outputs de comandos dos Container Nodes e acedê-los externamente a partir do host, utilizando a diretoria `/shared/`.

Para guardar o output dos comandos em ficheiros utilize os redireccionadores de output:

- `>` : Redirecciona a saída padrão (sobrescreve o ficheiro).
- `>>` : Redirecciona e anexa a saída a um ficheiro.
- `<` : Recebe a entrada de um ficheiro em vez do teclado.
- `2>` : Redirecciona as mensagens de erro para um ficheiro.
- `&>` : Redirecciona tanto a saída padrão como as mensagens de erro para um ficheiro.

Inicie o modo de execução e refaça o teste de débito com o comando `iperf3`, porém, agora no terminal de n1 execute o comando:

```
iperf3 -c <IP_n2> > /shared/iperf3-n1-n2.log
```

Verifique que tem acesso ao log a partir do host.



2. Wireshark

O Wireshark⁴ é um analisador de pacotes de código aberto que permite aos utilizadores capturar o tráfego numa rede de computadores e inspecionar os pacotes capturados, com suporte a centenas de protocolos de comunicação. Faça o download e instale o Wireshark no seu host: <https://www.wireshark.org/download.html>

2.1. Familiarização com o Wireshark

Nesta secção é realizada uma introdução ao Wireshark e a captura e análise de tráfego no sistema host. Abra o Wireshark na máquina para iniciar uma nova captura de tráfego. O Wireshark requer permissões de administrador para correr, portanto execute como administrador.

Observe que é possível abrir um ficheiro de uma captura anterior (**Open**) ou iniciar uma nova captura (**Capture**), como na Figura 4. Essa nova captura pode ser realizada nas diferentes interfaces de rede disponíveis no dispositivo utilizado (Ex.: Wi-Fi, Ethernet, dentre outras). Aqui partimos do pressuposto que estão num portátil ligado à Internet via Wi-Fi e vamos capturar o tráfego a partir dessa interface.

Selecione a interface Wi-Fi e clique no botão Start capturing packets, conforme demonstra a Figura 5.

Após o início da captura, será possível observar diversos pacotes a surgir no ecrã, correspondentes ao tráfego de rede em tempo real. Com a captura ainda em execução, aceda no browser ao seguinte endereço: <https://www.fccn.pt/>.

Para facilitar a análise do tráfego, pare a captura, com o botão “Stop capturing packets”. Além disso, o Wireshark permite a utilização de filtros para a busca de pacotes com informações específicas dos protocolos suportados, como o protocolo utilizado e os valores nos campos desse protocolo.

Nesse exemplo vamos verificar a existência de pacotes com protocolo TCP e que possuem o campo porta nesse protocolo com o valor 443, conforme a Figura 6. O campo filtro possui uma sintaxe própria, bastante abrangente, que permite o uso de expressões

⁴ <https://www.wireshark.org/>



lógicas, combinar expressões, o uso de arrays, etc. Para mais informações, verifique: https://www.wireshark.org/docs/wsug_html_chunked/ChWorkBuildDisplayFilterSection.html

Após aplicar o filtro, procure um pacote que contenha na coluna Info a informação “Client Hello (SNI=www.fccn.pt)”, conforme demonstra a Figura 7.

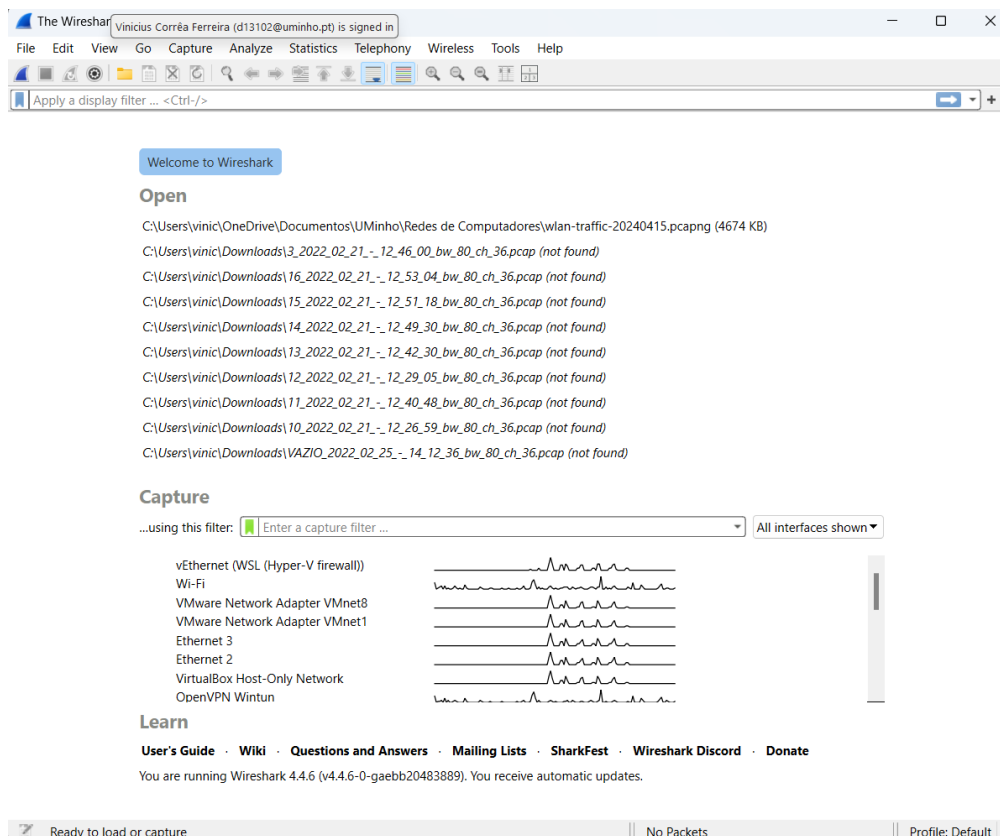


Figura 4 - Ecrã inicial do Wireshark

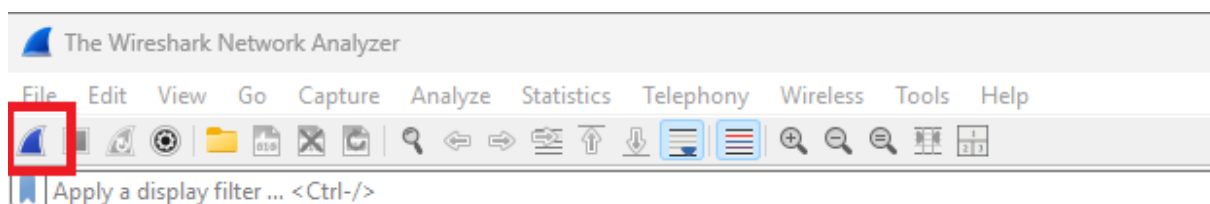


Figura 5 – Botão start capturing packets

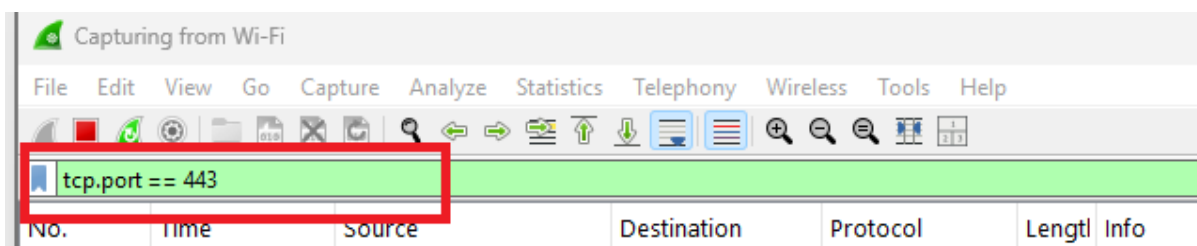


Figura 6 - Filtro do tráfego no Wireshark.

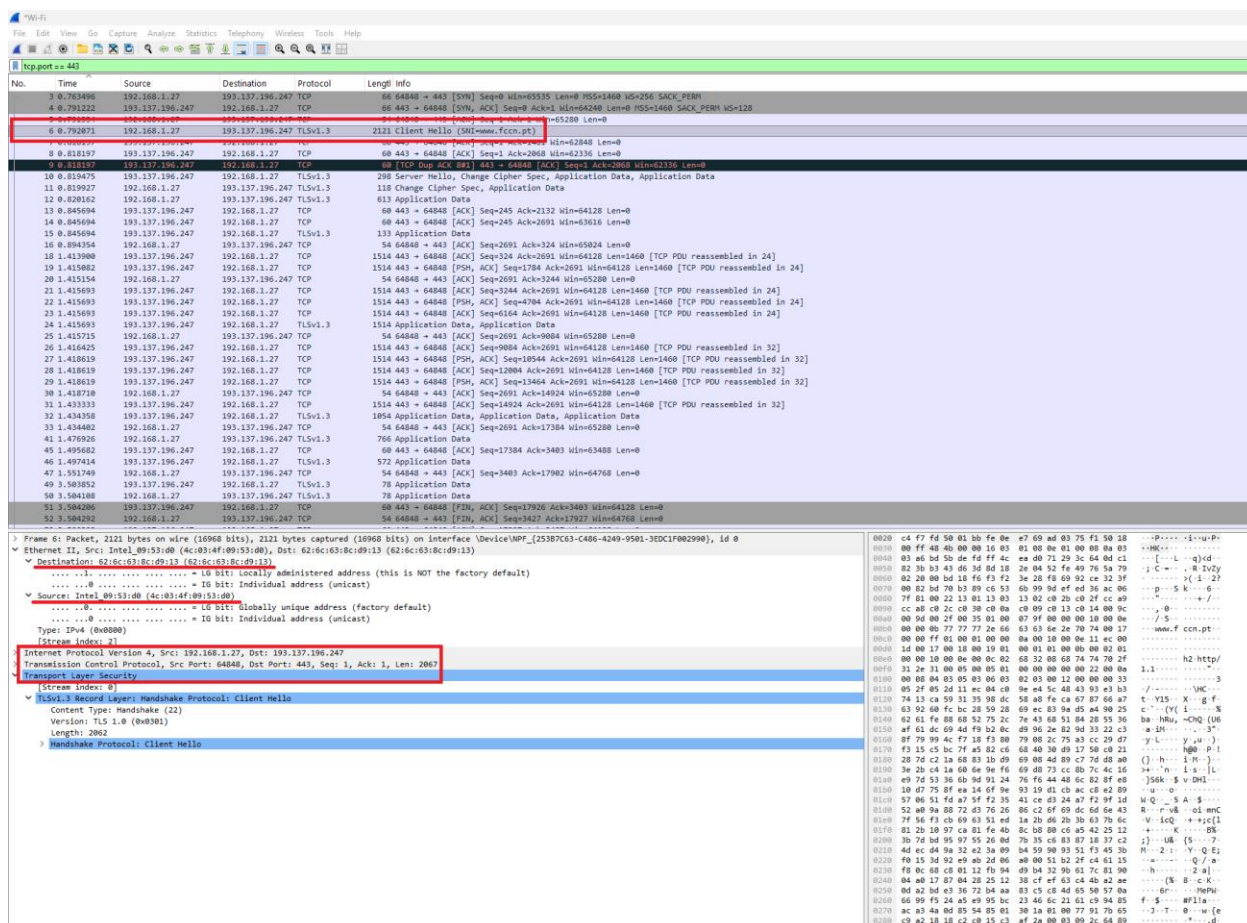


Figura 7 – Pacote com campo Info “Client Hello (SNI=www.fccn.pt)”.

Observe esse pacote e perceba que o Wireshark permite observar o encapsulamento protocolar e a pilha protocolar utilizada para transmitir/receber pacotes. Neste exemplo identificamos alguns que serão abordados mais adiante nesta UC, como o Ethernet, o Internet Protocol Version 4 (IPv4) e o Transmission Control Protocol (TCP). Explore a informação apresentada e tente encontrar os campos e valores de:

- o Endereço IP de origem
- o Endereço IP de destino
- o Porta de origem



o Porta de destino

2.2. Captura de Tráfego no CORE

Agora vamos realizar uma captura de tráfego no emulador CORE. Abra o CORE com a topologia salva no ficheiro `Exemplo-Ficha1.xml`. No CORE é possível realizar a captura em qualquer um dos nós do tipo **Container Nodes** em quaisquer de suas interfaces de rede.

Execute o comando `wireshark` no terminal do nó **n2** e inicie uma captura de tráfego na sua interface de rede. Após iniciar a captura, no terminal de **n1**, corra o comando: `ping -c 3 <IP_n2>`.

Após o comando terminar de executar, pare a captura e faça um filtro pelo protocolo `icmp`. Quantos pacotes foram filtrados? Qual a relação entre o parâmetro `-c` passado no ping e os pacotes capturados?

Após encerrar uma captura é possível salvá-la num ficheiro com a extensão `.pcap` ou `.pcapng`, o que permite que sejam abertas no Wireshark para posterior análise. Salve a captura na diretoria `/shared/` e verifique que pode abri-la a partir do host.

Para cenários que requerem a captura em diferentes nós, a gestão das múltiplas janelas de Wireshark pode se tornar complexa. Há uma ferramenta de captura de tráfego que corre em linha de comando, sem interface gráfica, denominada `tcpdump` que pode ser usada para esse fim.

Refaça o experimento anterior, mas ao invés de realizar a captura de tráfego utilizando o `wireshark`, corra o seguinte comando no terminal de **n2**:

```
tcpdump -i eth0 -w /shared/exemplo_captura_n2.pcap
```

Para encerrar um processo que corre no “foreground”, ou seja, que captura a interface de saída da linha de comandos, deve-se utilizar o atalho `Ctrl + C`.

Após correr o ping em **n1**, pare a captura, certifique-se que o ficheiro “`exemplo_captura_n2.pcap`” foi salvo na diretoria partilhada e a seguir abra a captura com o Wireshark. Verifique que os resultados são consistentes com os verificados anteriormente.



3. Exercícios

No formulário da FP1 envie um ficheiro zip com as respostas em pdf, os logs dos outputs dos comandos e os ficheiros de captura de tráfego necessários para realização dos exercícios. Nas respostas inclua todas as informações necessárias para a justificação, a partir de capturas do ecrã ou conteúdo dos outputs, e identifique os ficheiros utilizados para obter aquelas informações.

- 1) Crie uma topologia com dois nós PC e um router. Dê o nome dos PCs de PC1 e PC2, e do router de R1. A seguir, ligue ambos os PCs ao router. Remova os endereços IPv6 de todos os elementos. Configure os endereços IPv4 da seguinte maneira:
 - Na interface de PC1, que o liga a R1: 10.<PL>.<G>.20/24
 - Na interface de R1, que o liga a PC1: 10.<PL>.<G>.1/24
 - Na interface de PC2, que o liga a R1: 10.<PL+1>.<G>.20/24
 - Na interface de R1, que o liga a PC2: 10.<PL+1>.<G>.1/24
 - Onde PL é o número do turno PL e G é o número do grupo. Ex.: Se for o turno PL1 e grupo G08, os valores serão 1 e 8, respetivamente.
 - Salve essa topologia como TP1_<PL>_<G>.xml
- 2) Execute um comando ping e verifique se existe conectividade entre os PCs.
- 3) Configure ambas as ligações para uma largura de banda de 10 Mbps. Execute os comandos iperf3 e indique a largura de banda efetiva obtida.
- 4) Altere uma das ligações para uma largura de banda de 17 Mbps. Execute novamente os comandos iperf3 e indique a largura de banda efetiva. O valor obtido é igual a 17 Mbps? Justifique o valor obtido.
- 5) Altere a topologia de forma a obter uma largura de banda efetiva de aproximadamente 17 Mbps. Faça uma captura de tráfego do teste realizado para verificar a largura de banda efetiva.
- 6) Configure um atraso de 1 ms na ligação entre PC1 e R1. Execute um ping entre PC1 e PC2, qual é o atraso (RTT) medido? O valor é de 1 ms? Se não o for, justifique o seu valor.



- 7) Altere a topologia de forma a obter um atraso (RTT) de aproximadamente 4 ms. Faça uma captura de tráfego em simultâneo nos nós PC1 e PC2 e execute um ping com o parâmetro -c 5 a partir de PC1, destinado a PC2. Salve ambas as capturas. A partir da captura de tráfego realizada em **PC1**, responda:
- Identifique os endereços IP de origem e destino dos pacotes com o campo Info ping (Echo Request) e ping (Echo Reply). Compare os endereços obtidos entre os diferentes pacotes e relacione com os que foram configurados na topologia no CORE.
 - Nesses pacotes, há alguma informação no campo Info que é diferente entre Echo Request e Echo Reply? Se sim, quais?
 - Entre pacotes Echo Request e Echo Reply consecutivos, há alguma diferença no campo Info? Se sim, quais?
 - Compare as informações das alíneas anteriores obtidas a partir da captura realizada em PC1 com as obtidas na captura realizada em PC2, há diferenças? Quais?
- 8) Com os ficheiros da captura de tráfego dos exercícios 6 e 7. Faça um filtro `ip.addr == <IP_PC1>` e responda:
- Quais os protocolos utilizados no tráfego do comando iperf3?
 - Quais os protocolos utilizados no tráfego do comando ping?